

24

OpenCV

The essential toolkit for working with images and video

Load, change, draw on, and process pictures in code

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	A library for image and video processing
In code	<code>import cv2</code>
Best for	Loading, resizing, drawing, video frames
Works with	NumPy — images are just arrays
Pairs with	Your CNN and YOLO models

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is OpenCV?	The toolbox and what it's for
2	Images are NumPy Arrays	The core idea
3	Loading and Saving	Reading and writing images
4	The BGR Gotcha	OpenCV's color surprise
5	Common Operations	Resize, crop, color, blur
6	Drawing on Images	Boxes, circles, text
7	Working with Video	Frames and webcam
8	OpenCV + AI Models	The real workflow
9	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is OpenCV?

OpenCV is the most popular library for working with images and video in code. The name stands for Open Computer Vision. If a deep learning model is the brain that understands images, OpenCV is the hands that load, prepare, and handle them. It does the practical, everyday work around your AI models.

OpenCV: a big toolbox for working with images



Fig 1 — OpenCV is a big toolbox: load and save, resize, change colors, draw, find edges, handle video.

It can do simple things like opening a photo and resizing it, and advanced things like detecting edges or tracking objects in a video. In your projects, you use it to get images ready before they go into a CNN, and to draw the results (like detection boxes) back onto the image. You write it as `cv2` in code.

■ *OpenCV is fast and works everywhere — Python, on phones, on tiny devices. It pairs perfectly with your CNN and YOLO models, handling all the image-wrangling around them.*

2. Images are NumPy Arrays

Here's the key idea that makes everything click: in OpenCV, an image is just a NumPy array of numbers. You already learned NumPy in the very first topic, so you already know how to work with images. A color image is a 3D array: height x width x 3 color channels.

```
import cv2

img = cv2.imread('photo.jpg') # load an image

print(type(img))    # <class 'numpy.ndarray'> -- it's NumPy!
print(img.shape)    # (480, 640, 3) = height, width, 3 colors
print(img[0, 0])    # the color of the top-left pixel
```

Because images are arrays, everything you learned about NumPy works here: slicing to crop, indexing to read pixels, math to brighten. OpenCV just adds image-specific tools on top.

3. Loading and Saving

The two most basic operations: read an image from a file, and write one back. These are the bookends of almost every OpenCV program.

```
import cv2

# Load an image
img = cv2.imread('input.jpg')

# Check it loaded (None means the path was wrong)
if img is None:
    print('Could not load the image!')

# Get its size
height, width = img.shape[:2]
print(width, 'x', height)

# Save an image
cv2.imwrite('output.jpg', img)
```

■ If `cv2.imread` returns `None`, the file path is wrong or the file is missing. OpenCV does not raise an error here — it just returns `None` — so always check for it. This silent failure trips up many beginners.

4. The BGR Gotcha

This is the single most famous OpenCV surprise. Most tools store colors as RGB (Red, Green, Blue). But OpenCV stores them as BGR (Blue, Green, Red) — the order is reversed. If you load an OpenCV image and show it with another library, the colors look wrong: red and blue are swapped.

The #1 OpenCV gotcha: it uses BGR, not RGB

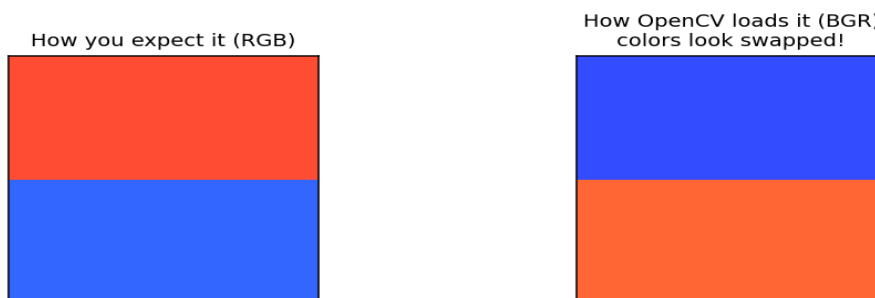


Fig 2 — OpenCV loads colors in BGR order. Show it elsewhere without converting and colors look swapped.

```
# OpenCV loads as BGR. To use with matplotlib or PyTorch,  
# convert to RGB first:  
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
  
# Convert to grayscale (black and white)  
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

■ *Remember this rule: OpenCV uses BGR. Whenever you move an image between OpenCV and another tool (matplotlib, PyTorch, PIL), convert the color order. Swapped red/blue colors is almost always this issue.*

5. Common Operations

These are the everyday operations you'll use constantly to prepare images for a model.

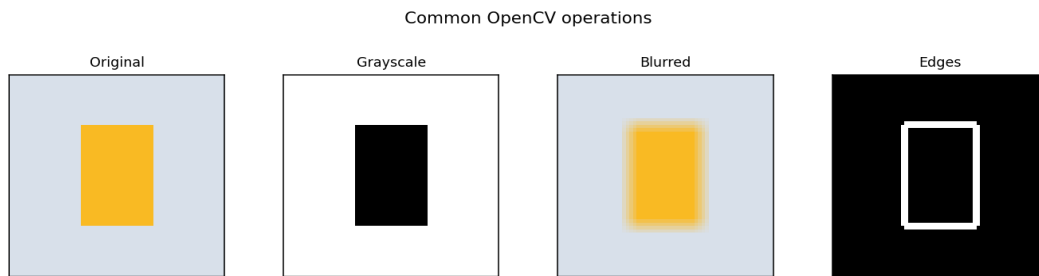


Fig 3 — A few common operations: grayscale, blur, and edge detection.

```
# Resize to a fixed size
resized = cv2.resize(img, (224, 224))

# Crop (it's just NumPy slicing!)
crop = img[50:200, 100:300] # rows 50-200, cols 100-300

# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Blur (smooth out noise)
blurred = cv2.GaussianBlur(img, (5, 5), 0)

# Edge detection (find outlines)
edges = cv2.Canny(img, 100, 200)

# Flip and rotate
flipped = cv2.flip(img, 1) # 1 = horizontal flip
rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
```

6. Drawing on Images

OpenCV can draw shapes and text directly onto an image. This is exactly how you show detection results — drawing a box around each found object with its label.

Drawing boxes, circles, lines, and text

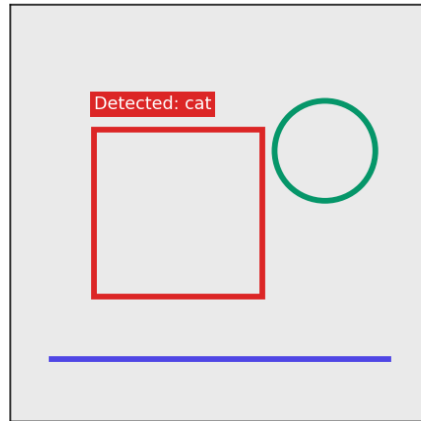


Fig 4 — Drawing a rectangle, circle, line, and text label onto an image.

```
# Draw a rectangle (box): image, top-left, bottom-right, color, thickness
cv2.rectangle(img, (50, 50), (200, 200), (0, 0, 255), 2)

# Draw a circle: image, center, radius, color, thickness
cv2.circle(img, (150, 150), 40, (0, 255, 0), 2)

# Draw a line
cv2.line(img, (0, 0), (300, 300), (255, 0, 0), 2)

# Write text
cv2.putText(img, 'cat 0.95', (50, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
```

■ Remember colors are BGR here too. (0, 0, 255) is RED, not blue. (255, 0, 0) is BLUE. This catches people out constantly when drawing.

7. Working with Video

A video is just a fast sequence of images called frames. OpenCV reads a video frame by frame, so you can process each frame like a normal image — perfect for running detection on live camera feeds.

```
import cv2

# Open a video file (or 0 for your webcam)
cap = cv2.VideoCapture('video.mp4')

while True:
    ret, frame = cap.read()    # read one frame
    if not ret:
        break                # video ended

    # 'frame' is a normal image - process it here
    # e.g. run your YOLO model, draw boxes...

    cv2.imshow('Video', frame) # show it
    if cv2.waitKey(1) == ord('q'): # press q to quit
        break

cap.release()
cv2.destroyAllWindows()
```

■ Use `cv2.VideoCapture(0)` for your webcam, or a filename for a video. This loop — read frame, process, show, repeat — is the basis of every real-time vision app, like live object detection.

8. OpenCV + AI Models

In real projects, OpenCV and your AI model work as a team. OpenCV handles the image work; the model does the understanding. Here's the typical flow for running detection on a video:

- OpenCV reads a frame from the video or camera.
- OpenCV resizes and converts it (BGR to RGB) for the model.
- The model (YOLO, CNN) makes its prediction.
- OpenCV draws the boxes and labels back onto the frame.
- OpenCV shows the result, then repeats for the next frame.


```
from ultralytics import YOLO
import cv2

model = YOLO('yolov8n.pt')
cap = cv2.VideoCapture(0) # webcam

while True:
    ret, frame = cap.read()
    if not ret: break
    results = model(frame) # detect
    annotated = results[0].plot() # draw boxes
    cv2.imshow('Live Detection', annotated)
    if cv2.waitKey(1) == ord('q'): break
```

9. Common Mistakes

Mistake	Why it's a problem	Fix
Forgetting BGR vs RGB	Colors look swapped	cv2.cvtColor before/after other tools
Not checking imread	Silent None, confusing errors	Check if img is None
Wrong color in drawing	(0,0,255) is red not blue	Remember BGR order
Wrong path to file	imread returns None	Use correct, full file path
Forgetting cap.release()	Camera stays locked	Release at the end
Confusing size order	resize is (width, height)	Note the (W, H) order

Quick Reference Cheatsheet

Task	Code
Import	import cv2
Load image	cv2.imread('photo.jpg')
Save image	cv2.imwrite('out.jpg', img)
Image size	img.shape # (H, W, channels)
Resize	cv2.resize(img, (224, 224))
Crop	img[y1:y2, x1:x2]
BGR to RGB	cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
Grayscale	cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
Blur	cv2.GaussianBlur(img, (5,5), 0)
Edges	cv2.Canny(img, 100, 200)
Flip	cv2.flip(img, 1)
Rectangle	cv2.rectangle(img, p1, p2, color, t)
Circle	cv2.circle(img, center, r, color, t)
Text	cv2.putText(img, text, pos, font, scale, color, t)
Open webcam	cv2.VideoCapture(0)
Read frame	ret, frame = cap.read()
Show window	cv2.imshow('name', img)

The one idea to remember: in OpenCV, images are just NumPy arrays. It's the toolbox that loads, prepares, and draws on images around your AI models. Watch out for BGR color order — that's the classic gotcha.

Next Topic: Artificial Intelligence — zooming all the way out to see the big picture: what AI really is, how machine learning and deep learning fit inside it, and where everything you've learned connects.